

dose_response_code_example.R

allanbusuulwa

2026-07-26

```
#####
# DOSE-RESPONSE ANALYSIS IN R
# Quantal (dead/alive) bioassay: raw counts -> LC50/LC90 -> figure.
# Example: Spinetoram vs. chilli thrips. The workflow is compound-agnostic.
#
# Reference: Ritz C, Baty F, Streibig JC, Gerhard D (2015). Dose-Response
# Analysis Using R. PLoS ONE 10(12): e0146021.
#
# Key idea: this is regression, not a lookup table. We fit a smooth dose ->
# mortality curve, then run it backwards to get doses we never tested.
#####

# --- 0. Setup -----

# suppressPackageStartupMessages() is separate from the knitr options above:
# startup chatter is emitted while this chunk runs, and drc is noisy because it
# masks dplyr::select and stats::gaussian.
suppressPackageStartupMessages({
  library(drc)      # drm(), ED(), mselect(), predict()
  library(ggplot2)
  library(dplyr)
  library(readxl)
})

theme_set(
  theme_bw(base_size = 14) +
    theme(
      text          = element_text(colour = "black"),
      axis.text     = element_text(colour = "black"),
      plot.background = element_rect(fill = "white"),
      panel.background = element_rect(fill = "white")
    )
)

# --- 1. Data -----
# Expected columns: conc (ppm), dead, total, rep

dt <- read_excel("data/Spinetoram_baseline_screen1_(09-April-2026).xlsx") |>
  mutate(prop_dead = dead / total)

glimpse(dt)
```

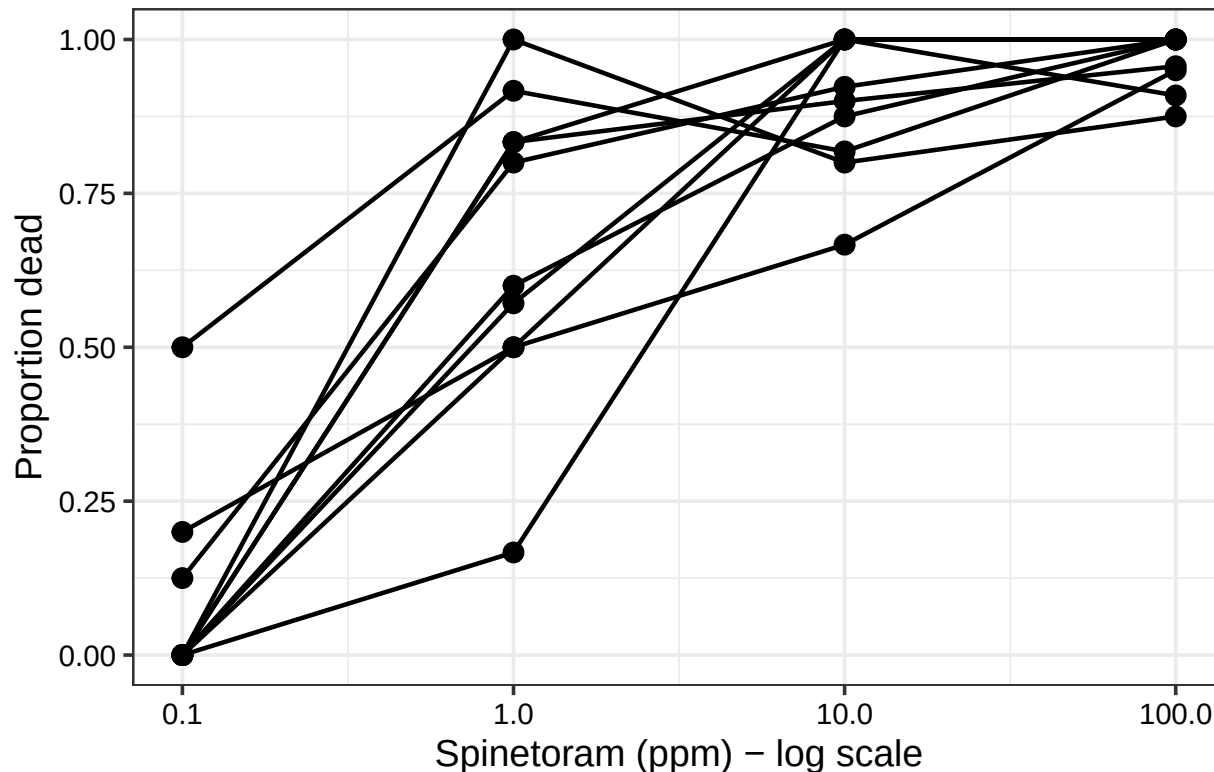
```
## Rows: 50
## Columns: 5
## $ conc      <dbl> 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.1, 0.1, ~
## $ rep       <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10~
## $ total     <dbl> 8, 4, 5, 4, 10, 3, 4, 6, 4, 5, 4, 4, 10, 6, 10, 10, 5, 10, 8~
## $ dead      <dbl> 0, 0, 1, 2, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 2, 0, 0, 1, 3, ~
## $ prop_dead <dbl> 0.0000000, 0.0000000, 0.2000000, 0.5000000, 0.1000000, 0.000~

# Response type decides the likelihood. Ours is aggregated binary (dead out of
# total) -> binomial.
#
# TRAP: prop_dead is for plotting only. Never fit to it - a proportion discards
# sample size, treating 1/2 dead as equal evidence to 50/100.

# --- 2. Look before you fit -----
# Fitting will succeed on nonsense. Check: monotone? replicates agree? does the
# curve actually turn, or is everything pinned at 0 and 1?
#
# Log axis because doses are a geometric series - potency is about ratios, not
# differences. (Same reason the models below are *log*-logistic.)
#
# TRAP: log(0) is undefined, so the untreated control is dropped for PLOTTING
# ONLY. drm() handles dose 0 natively and keeps it in the fit.

ggplot(filter(dt, conc > 0), aes(x = conc, y = prop_dead, group = rep)) +
  geom_point(size = 3) +
  geom_line(linewidth = 0.8) +
  scale_x_log10() +
  labs(title = "Raw mortality by replicate",
       x = "Spinetoram (ppm) - log scale",
       y = "Proportion dead")
```

Raw mortality by replicate



```
# --- 3. The four parameters -----
# Every drc curve uses the same letters:
#   b = slope, c = lower asymptote, d = upper asymptote, e = EC50
#
# The number in the model name is how many are ESTIMATED; the rest are fixed:
#   LL.2 fixes c = 0, d = 1      LL.3 fixes c = 0      LL.4 fixes nothing
# LL.2() is literally LL.4(fixed = c(NA, 0, 1, NA)).
#
# Fixing is a biological claim, not a convenience. d = 1 says a high enough dose
# kills everything; c = 0 says nothing dies untreated. If your controls show
# natural mortality, c = 0 is false - estimate it.
#
# LL2.x vs LL.x: identical curves, identical AIC. LL2.x estimates e = log(EC50)
# instead of e = EC50, so confidence intervals are built on the log scale and
# back-transformed - asymmetric and strictly positive, as a concentration must
# be. The paper recommends this for small datasets (<15-20 points).
#
# TRAP: e is on the LOG scale in summary(). exp() it for ppm.

# --- 4. Fit candidate models -----
# Fit three because we don't know a priori whether Spinetoram reaches 100%
# mortality at achievable doses. Let AIC arbitrate.
#
# dead/total ~ conc plus weights = total gives drm the counts, not a bare
# proportion. type = "binomial" switches from least squares to binomial ML -
```

```
# this is the line that makes it a quantal bioassay.
```

```
m1 <- drm(dead/total ~ conc, weights = total, data = dt,
  fct = LL2.2(names = c("Slope", "log(EC50)")),
  type = "binomial")

m2 <- drm(dead/total ~ conc, weights = total, data = dt,
  fct = LL2.3(names = c("Slope", "Upper Asymptote", "log(EC50)")),
  type = "binomial")

m3 <- drm(dead/total ~ conc, weights = total, data = dt,
  fct = LL2.4(names = c("Slope", "Lower Asymptote",
    "Upper Asymptote", "log(EC50)")),
  type = "binomial")

summary(m1); summary(m2); summary(m3)
```

```
##
## Model fitted: Log-logistic (log(ED50) as parameter) with lower limit at 0 and upper limit at 1 (2 pars)
##
## Parameter estimates:
##
##              Estimate Std. Error  t-value p-value
## Slope:(Intercept)    -0.838238   0.080013 -10.4762 < 2e-16 ***
## log(EC50):(Intercept) -0.352688   0.198180  -1.7796 0.07514 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

##
## Model fitted: Log-logistic (log(ED50) as parameter) with lower limit at 0 (3 parms)
##
## Parameter estimates:
##
##              Estimate Std. Error t-value  p-value
## Slope:(Intercept)    -1.285477   0.272585 -4.7159 2.407e-06 ***
## Upper Asymptote:(Intercept) 0.954576   0.019012 50.2082 < 2.2e-16 ***
## log(EC50):(Intercept)    -0.645114   0.202015 -3.1934 0.001406 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

##
## Model fitted: Log-logistic (log(ED50) as parameter) (4 parms)
##
## Parameter estimates:
##
##              Estimate Std. Error t-value  p-value
## Slope:(Intercept)    -1.154514   0.396548 -2.9114 0.003598 **
## Lower Asymptote:(Intercept) -0.040279   0.134660 -0.2991 0.764849
## Upper Asymptote:(Intercept) 0.959646   0.023294 41.1978 < 2.2e-16 ***
## log(EC50):(Intercept)    -0.710378   0.359944 -1.9736 0.048429 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Note: no starting values needed - drc's self-starters derive them from the
# data (getInitial() shows what was used).
```

```
# --- 5. Model selection -----
# AIC = fit, minus a charge per parameter. Lower is better; a gap of ~2+ is a
# real difference. A more flexible model always fits better - AIC asks whether
# the extra flexibility paid for itself.
```

```
AIC(m1, m2, m3)
```

```
##      df      AIC
## m1  2 494.4709
## m2  3 489.6578
## m3  4 490.8752
```

```
model_list <- list(LL2.2 = m1, LL2.3 = m2, LL2.4 = m3)
best_name  <- names(model_list)[which.min(sapply(model_list, AIC))]
best_model <- model_list[[best_name]]
```

```
cat("Best model by AIC:", best_name, "\n")
```

```
## Best model by AIC: LL2.3
```

```
# Is log-logistic the right SHAPE? Weibull I is steep early with a long tail;
# Weibull II is the mirror. Log-logistic is symmetric on the log-dose scale -
# an assumption worth testing, since asymmetry changes the LC90.
# With few dose levels these often can't be separated. Run it to know, not to
# go shopping for a better number.
```

```
mselect(best_model, list(W1.2(), W2.2()))
```

```
##      logLik      IC Lack of fit
## W1.2 -241.1973 486.3945 2.451843e-03
## LL2.3 -241.8289 489.6578 1.046698e-03
## W2.2 -255.0727 514.1453 5.653905e-06
```

```
# Is there a dose-response at all? This is the test people think summary()'s
# p-values give them. It isn't. This is.
```

```
noEffect(best_model)
```

```
## Chi-square test      Df      p-value
##      -44.86474      2.00000      1.00000
```

```
# --- 6. LC50 and LC90 -----
# Inverse regression: run the fitted curve backwards. ED() does this after
# fitting (delta method for the SEs), so you fit once and ask as many questions
# as you like.
```

```
#
```

```
# RELATIVE vs ABSOLUTE - the distinction that catches everyone:
```

```
#   relative (default) = % change between the curve's own lower and upper limits.
```

```
#                               If the curve tops out at 0.955, "ED90" = 0.86 mortality.
```

```
#   absolute           = the dose reaching 0.9 mortality on the real probability
#                               scale. For "what kills 90% of thrrips?", use this.
```

```
#
```

```
# TRAP: absolute levels are on the RESPONSE scale - 0.9, not 90. Asking for 90
```

```
# requests 9000% mortality and returns NaN with a cryptic log() warning.
```

```
#
```

```
# TRAP: an absolute LC90 only exists if the ceiling clears 0.9. If the compound
```

```
# tops out at 85%, NaN is the correct answer, not a bug.
```

```

#
# interval = "fls" forms the interval on the log scale and back-transforms.
# ("delta" is symmetric in ppm and can go negative; "fieller" suits ratios.)

# Change to #+ effective-doses, warning = TRUE while developing. ED() emits a
# log() warning alongside NaN when the ceiling doesn't clear the level asked
# for - with a ~95% top dose that is a live possibility for the LC90, and the
# warning is the analysis telling you something.
lc <- ED(best_model, c(0.5, 0.9), interval = "fls", type = "absolute")

##
## Estimated effective doses
##
##      Estimate      Lower      Upper
## e:1:0.5  0.56495  0.37525  0.85055
## e:1:0.9  4.64246  1.50902 14.28247
lc

```

	Estimate	Lower	Upper
e:1:0.5	0.56495	0.37525	0.85055
e:1:0.9	4.64246	1.50902	14.28247

```

lc

```

	Estimate	Lower	Upper
e:1:0.5	0.5649476	0.3752472	0.8505481
e:1:0.9	4.6424644	1.5090155	14.2824749

```

# TRAP: ED() returns rows in the order requested, so positional indexing must
# match the levels vector above. Change one, change the other.
lc50 <- lc[1, "Estimate"]
lc90 <- lc[2, "Estimate"]

cat("LC50 =", round(lc50, 3), "ppm | LC90 =", round(lc90, 3), "ppm\n")

## LC50 = 0.565 ppm | LC90 = 4.642 ppm

# On the summary() p-values: each tests H0: parameter = 0.
# e (log EC50): "LC50 is at 1 ppm" - an arbitrary reference. Not interesting.
# b (slope): "curve is flat" - meaningful, but noEffect() tests it properly.
# d (upper): "ceiling is zero mortality" - certain to be rejected.
# The stars are not a fit diagnostic. Report the STANDARD ERROR; that's what
# propagates into the interval around your LC50.

# --- 7. Plot the fitted curve -----
# Evaluate the model on a fine log-spaced grid. A linear seq() would waste most
# of its points where the curve is flat.
#
# TRAP: must be a data.frame, not a tibble. drc's predict() does newdata[, 1],
# which returns a vector from a data.frame but a one-column list from a tibble.

conc_range <- range(dt$conc[dt$conc > 0])
dose_grid <- data.frame(
  conc = 10^seq(log10(conc_range[1]), log10(conc_range[2]), length.out = 400)
)

# Confidence BAND, not error bars - the uncertainty belongs to the fitted curve.
pred <- predict(best_model, newdata = dose_grid, interval = "confidence")
dose_grid$fit <- pred[, 1]
dose_grid$lower <- pred[, 2]

```

```

dose_grid$upper <- pred[, 3]

resp <- ggplot() +
  geom_ribbon(data = dose_grid, aes(x = conc, ymin = lower, ymax = upper),
            fill = "steelblue", alpha = 0.2) +
  geom_line(data = dose_grid, aes(x = conc, y = fit),
            colour = "steelblue", linewidth = 0.9) +

  # Vertical jitter only. width = 0 keeps points at their true concentration -
  # horizontal jitter on a log axis would misstate the data.
  geom_jitter(data = filter(dt, conc > 0), aes(x = conc, y = prop_dead),
             size = 2.5, colour = "black",
             width = 0, height = 0.02, alpha = 0.6) +

  # These labels should sit ON the curve. If one floats off, you're probably
  # looking at relative rather than absolute ED values.
  annotate("text", x = lc50, y = 0.5,
          label = paste0("LC50 = ", round(lc50, 2), " ppm"),
          colour = "firebrick", hjust = -0.1, size = 4, fontface = "bold") +
  annotate("text", x = lc90, y = 0.9,
          label = paste0("LC90 = ", round(lc90, 2), " ppm"),
          colour = "darkorange", hjust = -0.1, size = 4, fontface = "bold") +

  scale_x_log10() +
  scale_y_continuous(breaks = seq(0, 1, by = 0.2)) +

  # TRAP: zoom with coord_cartesian(), not scale limits. Scale limits DELETE
  # points outside the range - jittered points at exactly 0 or 1 would vanish.
  coord_cartesian(ylim = c(-0.03, 1.03)) +

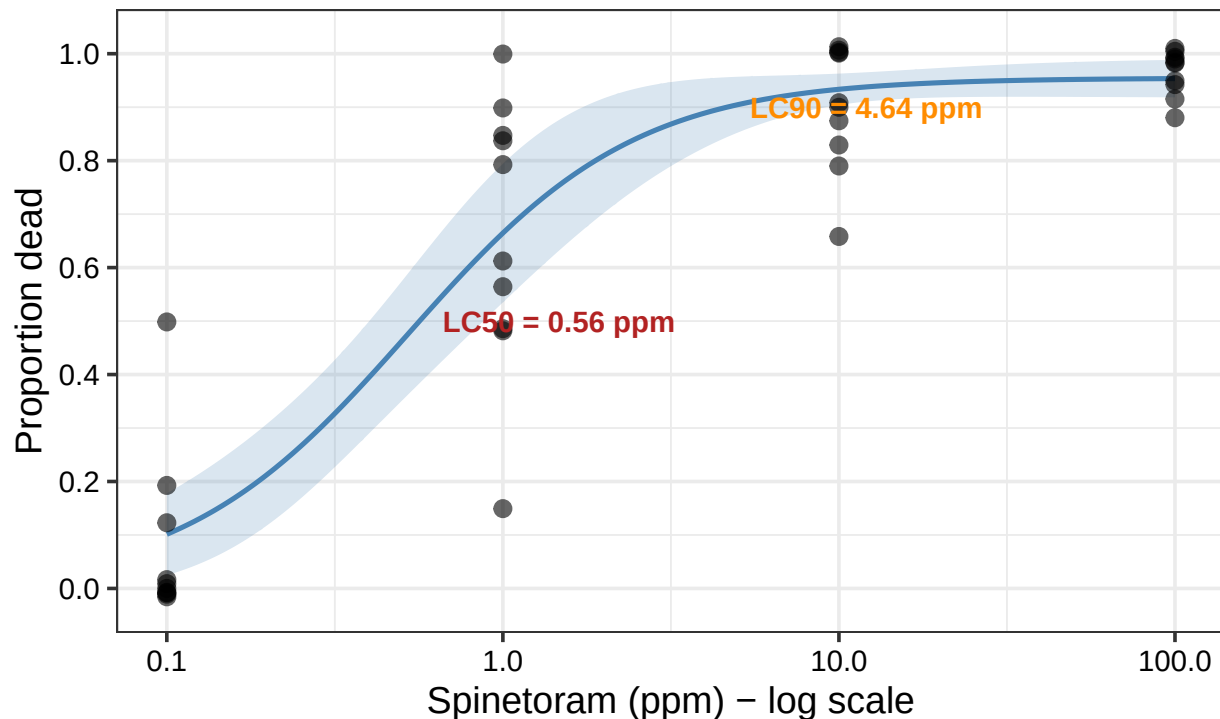
  labs(title = "Spinetoram dose-response curve",
       subtitle = "Chilli thrips lab colony baseline (09-April-2026)",
       x = "Spinetoram (ppm) - log scale",
       y = "Proportion dead")

resp

```

Spinetoram dose-response curve

Chilli thrips lab colony baseline (09-April-2026)



```
dir.create("graphs", showWarnings = FALSE)
ggsave("susceptible_colony_drc.png", plot = resp, path = "graphs",
       width = 10, height = 6, dpi = 800)

# --- 8. Where this goes next -----
# A baseline LC50 becomes science when a second curve is compared to it.
#
# Fit two curves at once - pmodels sets which parameters are shared:
# m_both <- drm(dead/total ~ conc, colony, weights = total, data = dt_both,
#               fct = LL2.3(), type = "binomial",
#               pmodels = list(~colony-1, ~1, ~colony-1))
#
# Do they differ? Fit the single-curve model and compare with anova()
# (likelihood-ratio chi-square for binomial fits).
#
# Resistance ratio = relative potency = ratio of effective doses:
# EDcomp(m_both, c(50, 50), interval = "delta")
# Report the interval. "RR = 4.2" is much weaker than "RR = 4.2 (95% CI
# 2.1-8.4)", and if that interval covers 1 you haven't shown resistance.
# Relative potency is only constant across levels for PARALLEL curves - if
# slopes differ, RR at LC50 != RR at LC90 (see relpot()).
#
# Compare slopes: compParm(m_both, "b", "-")
```



```

# --- Limitations -----
# 1. INDEPENDENCE. Insects sharing an arena or cohort aren't independent, so
#    SEs are too small and intervals too narrow. See medrc for mixed effects.
# 2. THIS LC90 IS FRAGILE. The top dose reaches ~95%, so 90% sits right against
#    an estimated ceiling. The fix is a higher top dose, not more statistics.
# 3. MODEL SELECTION ISN'T FREE. We chose a model from the data, then reported
#    intervals as if it were fixed in advance. See maED() for model averaging.
# 4. AIC RANKS, IT DOESN'T VALIDATE. Check the curve against the points and use
#    modelFit(). The plot is a diagnostic, not decoration.
#####

```